Michelle T. George

Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal
Turn a rough idea into a precise, buildable project description.

Instructions
Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title:
   Quiz App (Student quiz & analyzing score system)
2. Problem you want to solve (2–3 short sentences):
   To help teachers to store quiz scores and analyze student performance
3. Target users (who will actually use this?):
   Teachers
4. Inputs (what data / actions will the program receive?):
   - Student name
   - Marks scored by each student
5. Outputs (what will the user see / get?):
   - Total Students and the marks they got
   - Calculation of average marks, highest & lowest marks
6. Constraints or assumptions (keep it realistic):
   - Data is entered manually by the teacher
   - Number of student is limited (eg: upto 50)
   - Marks are valid numbers (0-100)

Part B – Core features vs "nice-to-have"

1. List minimum features needed for a working version (MVP):
   - Add students name and marks
   - Calculate average marks
   - Display highest & lowest marks
2. List optional features (only if time allows):
   - Grade classification based on marks
   -
3. Final question:
   - If you had only 1 week, which ONE feature would you absolutely keep? Why?

I would keep the feature to add student score and calculate average marks, because it is the most important functionality. It helps teachers quickly understand overall class performance.

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal
Learn to split a project into logical modules and functions.

Instructions
Use your project from Activity A1.

Project Quiz Score

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):
   - Module 1: Data input -> Student name, marks entered
   - Module 2: Processing -> functions (average, highest, lowest marks)
   - Module 3: Reporting (Result will be shown)
   - Module 4: Storage -> Save student data.
2. For each module, write its main responsibility in one sentence:
   - Module 1 does: Collect user input data
   - Module 2 does: Perform calculations.
   - Module 3 does: Display the results clearly to the users
   - Module 4 does: Store and manage student data for future use.

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: To find average scores of students in an exam (Module 1 -> to give input of data)
2. Functions planned:
   - Function 1 name: Calculation of average scores (based on data)
     - Processing
     - Input(s): Giving the list of students marks (Input list)
     - Output: Calculated average will be given
   - Function 2 name: To find highest score
     - Input(s): list (75, 80, 95, 90)
     - Output: int (Highest mark (95))
   - Function 3 name: To find lowest score
     - Input(s): list [75, 80, 95, 90]
     - Output: int (lowest mark (75))
3. Draw or briefly describe how data flows between these functions:

Input (Student names & marks) -> stored as list -> Processing functions (average, highest, lowest) -> Results generated -> Reporting module displays output

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal
Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions
Consider your project's core data (e.g. patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

   Transaction Patient record.

2. For this entity, list required fields:

   - Field 1: Name (type: int/str/float/...)
   - Field 2: ~~Amount~~ Transferred (type: ~~int~~ float)
   - Field 3: ~~Bank Name~~ Date of appointment (type: str)
   - Field 4: Slot attended (type: bool.)

   Patient Name : str
   Date of Appointment : int
   Doctor's Name : str
   Slot Allowded : ~~Mean~~ int.

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

python

```
item = {
  "field1": Name : Michelle
  "field2": Amount transferred : 75,000
  ...
}
```

2. Option B – Class:

python

```
class Item: Transaction
  def __init__(self, field1, field2,...): name amount transferred self
    self.field1 = field1 name name
    self.field2 = field2 amount amount
    ...
```

s1 = Transaction("Michelle", 75,000)
print(s1.name, s1.amount transferred)

3. Which option will you use, and why?

   Class → for a project with many transactions, a class is better. It gives a fixed structure, prevent typos in field names, allows adding methods like calculate date and make code cleaner.

4. Write a short example of creating and using this structure:

python

```
class Transaction:
  def __init__(self, name, amount, date, slot_attended):
# Example creation and simple usage
    self.name = name
    self.amount = amount
    self.date = date
    self.slot = attended = slot_attended
def show_details(self):
  status = "attended" if self.slot_attended else "Missed"
  print(f"{self.name} paid {self.amount} on {self.date}. Status: {status}")
```

Activity A4 – Planning Tests Before Coding

Title: Think in Test Cases (TDD Lite)

Goal
Build habit of designing tests before writing code.

Instructions
Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: fee calculation
2. In simple words, what should this function do?
   This should find students who have to pay regular fee no fee scholarship etc
3. List at least 5 test cases (inputs and expected outputs):

Test #	Input(s)	Expected output
1	If 10,000, "regular"	10,000
2	10000 "scholarship"	8,000
3	10,000 "staff child"	7000
4	0, "regular"	0
5	20,000 "invalid"	-1

4. Now write a small test harness:

python

```
def my_func(...): calculate_fee (base_fee, category):
    # TODO: implement    If category == "regular"
                             return base_fee
    pass                 elif category == "scholarship":
                             return base_fee * 0.8
                         elif category == "staff child"  return base fee * 0.7  else :
                                                                                return -1
# Tests                  calculate fee (10 000 " regular") "expected:", 10,000
print("Test 1:", my_func(...), "expected:", ...)
                         calculate fee (10,000 " scholarship") "expected:", 8,000
print("Test 2:", my_func(...), "expected:", ...)
```

5. Reflection:
   - Did writing tests first change how you thought about the function? How?

   Yes, It was little but easy to understand.

Activity A5 – Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal
Identify and protect weak points with validation and exception handling.

Instructions
Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: Invalid input
   - How to detect it? wrong value as input
   - How to handle it politely for the user? Ask again to give input
2. Failure Point 2: File not save error
   - Detect: File not found
   - Handle: Check again and save
3. Failure Point 3: Network Issues
   - Detect: Program not running
   - Handle: Check the network availability.

Part B – Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")


try:
  with open(file_path, "r") as f:
    data = f.read()
  print("File loaded successfully.")
except FileNotFoundError:
  print("File not found. Please check the path.")
except PermissionError:
  print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

```
# Your robust code:  file_path = input ("Enter file path:")
                     while True:
                         try:
                             with open (file_path, "r") as f:
                                 data = f.read ()
                             print ("file loaded successfully.")
                             break
                         except File not found Error:
                             print ("file not found. Please check the path.")
                             file_path = input ("Enter file path again:")
```

Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal
Practice cleaning up messy, working code into clear, modular code.

Instructions
Take one working script (from your own work or given by teacher).
It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:
grade calc.py - Take marks for 5 subjects, calculates total, average and assigns grade A|B|C|F

2. Identify at least three problems (long function, repetition, confusing names, etc.):
   - Problem 1: Repetition - Same input ( ) line copied 5 times instead of using a loop
   - Problem 2: Confusing name s1, s2, t, a, g don't explain what they store
   - Problem 3: No functions | not reusable - can't calculate grades for another student without rerunning everything

3. Plan refactoring steps (high level):
   - Step 1: Use a loop and list to collect marks instead of 5 separate variables
   - Step 2: Extract logic into functions: calculate_total(), calculate_average()
   - Step 3: Rename variables to marks, total, average, assign_grade()

4. After refactoring, write or paste your improved version here: average, grade and add input validation

python

 # Refactored code:

5. Reflection:
   - What became easier to understand after refactoring?

Everything. Each function now does one job with a clear name, so I can read assign_grade(average) and instantly know what it does. Using a loop means I can change to 6 subjects by editing one number instead of copying code. The validation also stops crashes if someone types "abc" by mistake. If the grading rules change later, I only edit assign_grade() instead of hunting through the whole file. The code went from "only the person who wrote it understands it" to anyone can maintain it."

Activity A7 – Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line.

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:
   Example:
   - python student_tool.py add --name Alice --score 85
2. What should the tool do in ONE sentence?

Add a student's name and score to a record using command-line arguments

3. Sketch CLI arguments:

Argument / Option	Meaning
add	subcommand that tells the tool to add a new student
--name	The student's name, type string
--score	The student's score, type integer

4. Implement with argparse (skeleton):

python

```
import argparse


parser = argparse.ArgumentParser(description="Describe your tool here")

parser.add_argument("--name", type=str, help="Student name")

parser.add_argument("--score", type=int, help="Student score")

args = parser.parse_args()


# Use args.name, args.score here
Alice
print("Name:", args.name)
85
print("Score:", args.score)
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

Python student_tool.py add --name Alice --score 85

```
Python import argparse
parser = argparse.ArgumentParser(description="Student record tool")
parser.add_argument("--name", type=str, help="student name")
parser.add_argument("--score", type=int, help="Student score")
args = parser.parse_args()
```

Activity A8 – Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

Goal

Produce a realistic plan and self-review for your project.

Part A – Milestones and Timeline

1. List 4–6 milestones (each should be testable):
   - Milestone 1: Core grading logic works on sample CSV
   - Milestone 2: CLI accepts answer key + student responses
   - Milestone 3: Generates score report as CSV with error handling
   - Milestone 4: Add timing / progress bar and test on 500 student dataset
   - Milestone 5: Package with - help docs and demo script
2. For each milestone, define:

Milestone	Concrete Deliverable	Due Date
Milestone 1	grade.py. run that takes 2 files &	2
Milestone 2	exam - speed - to of.py with argparse takes key & ~~Responses~~	4
Milestone 3	Running tool creates result.csv Catches file not found error with clear msg	6
	Process 500 row in < 3 sec Show progress running & done	8
	README, md with examples + demo.sh that runs end to end	10

Part B – Risk & Mitigation

1. Identify 2–3 biggest risks (technical or time):
   - Risk 1: Answer key file format is inconsistent from teachers.
     - Plan B if this fails: Support both CSV and excel, add format flag, and create a clear template file
   - Risk 2: Script is fast but school PC's don't have python installed
     - Plan B if this fails: Package as a single .exe using pyinstaller so no install needed
   - Risk 3: Running out of time for the progress bar / UX features
     - Plan B if this fails: Ship core grading first. Progress bar is "nice to have" for v2

Part C – Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

   Breaking the problem into tiny functions helped

2. One thing I now believe about myself as a problem-solver:

   I am good at turning a vague request like "make it faster" into concrete testable steps. I won't just write code. I'll first ask what exactly is slow.

3. If I start a new project tomorrow, what will I do differently from day 1?

   Starts with milestone tables on day 1